

CSEN604: Data Bases II

Mohamed Karam Gabr

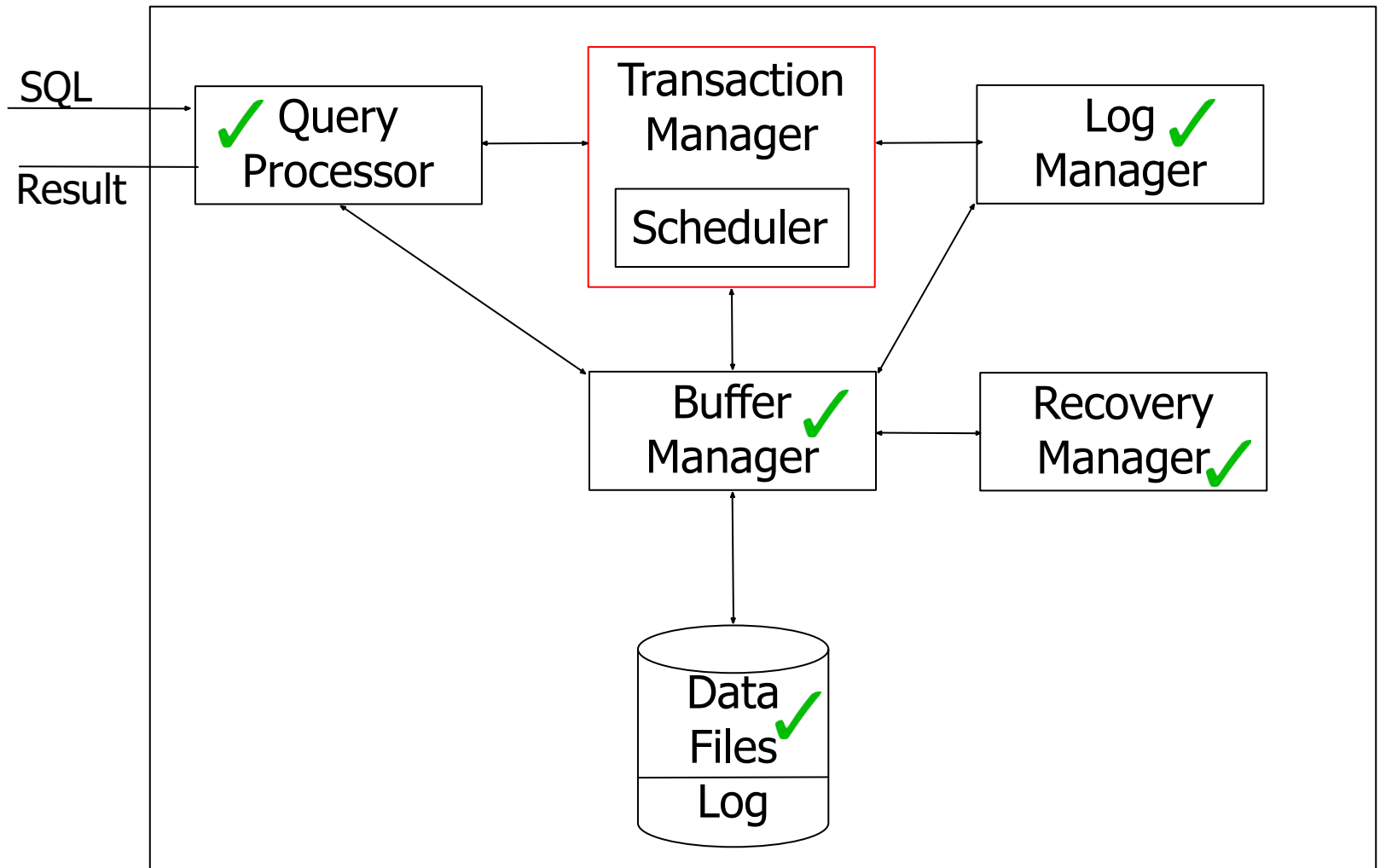
Transactions

Spring 2025

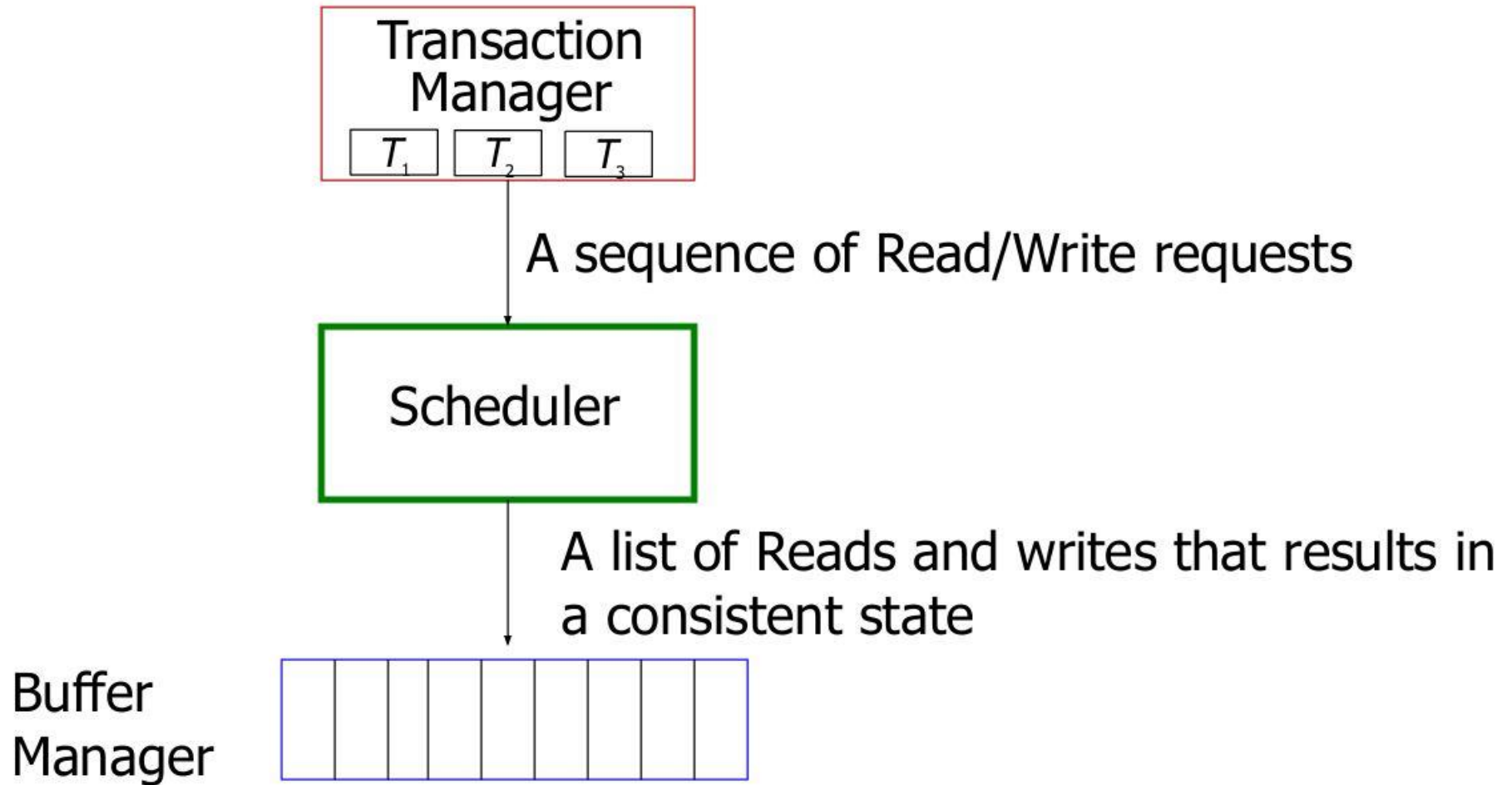
Outline

- Transactions.
- Schedules.
- Serializable.
- Conflict Serializable.
- Conflict Equivalent.
- Precedence Graph.
- Locks.
- Locking Trees.

Introduction

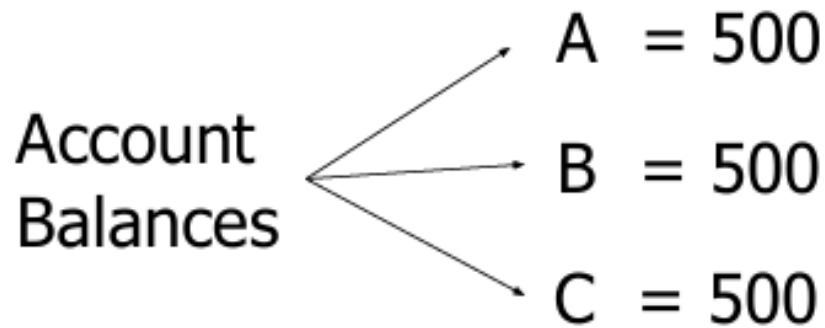


Introduction



Example

Bank database: 3 Accounts

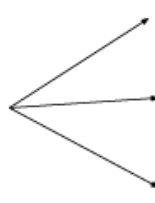


Property: $A + B + C = 1500$

Money should not leave/enter

Example

Account Balances



A = 500
B = 500
C = 500

Property: $A + B + C = 1500$

Transaction T1: Transfer 100 from A to B

A = 500, B = 500, C = 500

→ Read (A, t)

t = t - 100

Write (A, t)

Read (B, t)

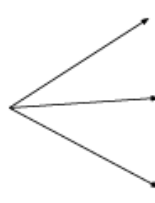
t = t + 100

Write (B, t)

A = 400, B = 600, C = 500 →

Example

Account Balances



A = 500
B = 500
C = 500

Property: $A + B + C = 1500$

Transaction T2: Transfer 100 from A to C

A = 500, B = 500, C = 500



Read (A, s)

$s = s - 100$

Write (A, s)

Read (C, s)

$s = s + 100$

Write (C, s)

A = 400, B = 500, C = 600



Schedule 1:

Transaction T1	Transaction T2	A	B	C
Read (A, t)		500	500	500
t = t - 100				
Write (A, t)		400	500	500
Read (B, t)				
t = t + 100				
Write (B, t)		400	600	500
	Read (A, s)			
	s = s - 100			
	Write (A, s)	300	600	500
	Read (C, s)			
	s = s + 100			
	Write (C, s)	300	600	600

$$300 + 600 + 600 = 1500$$

Schedule 2:

Transaction T1	Transaction T2	A	B	C
Read (A, t) $t = t - 100$ Write (A, t) Read (B, t) $t = t + 100$ Write (B, t) Read (C, s) $s = s + 100$ Write (C, s)		500	500	500
		400	500	500
	Read (A, s) $s = s - 100$ Write (A, s)	300	500	500
		300	600	500
		300	600	500
		300	600	600

$$300 + 600 + 600 = 1500$$

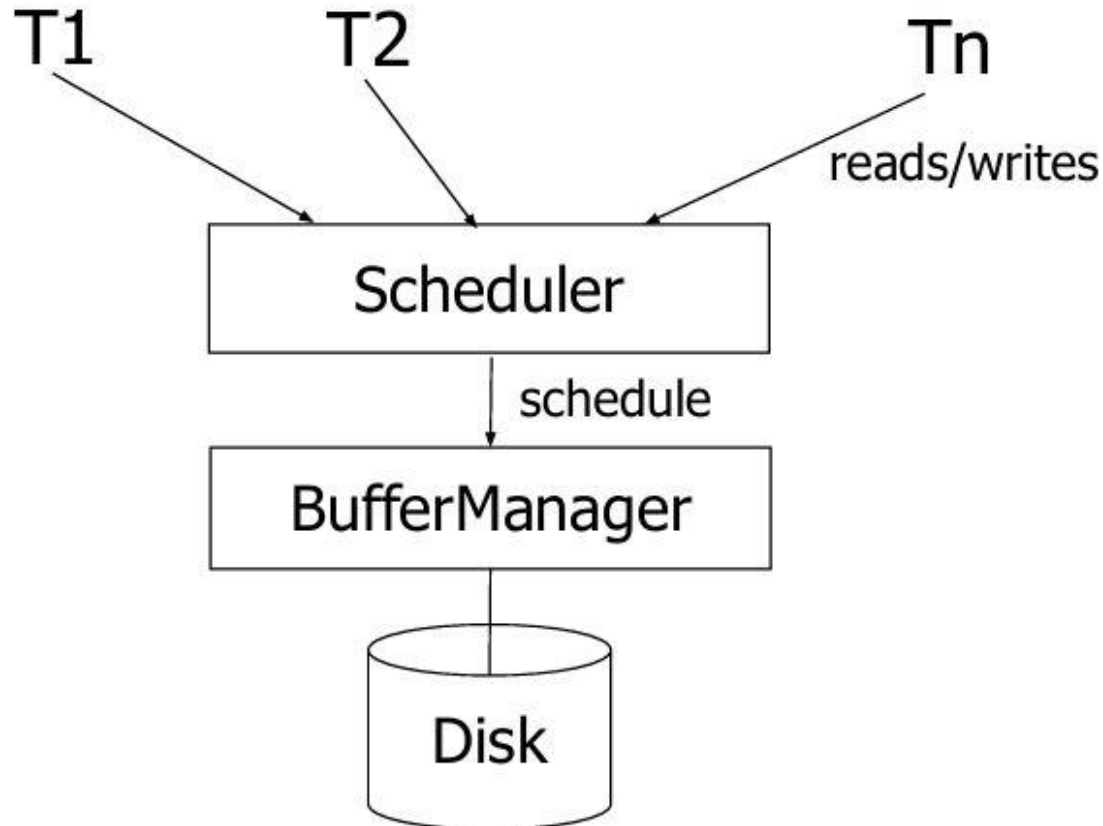
Schedule 2:

Transaction T1	Transaction T2	A	B	C
Read (A, t) $t = t - 100$		500	500	500
	Read (A, s) $s = s - 100$ Write (A, s)	400	500	500
Write (A, t) Read (B, t) $t = t + 100$ Write (B, t)		400	500	500
		400	600	500
	Read (C, s) $s = s + 100$ Write (C, s)	400	600	600

$$400 + 600 + 600 = 1600$$

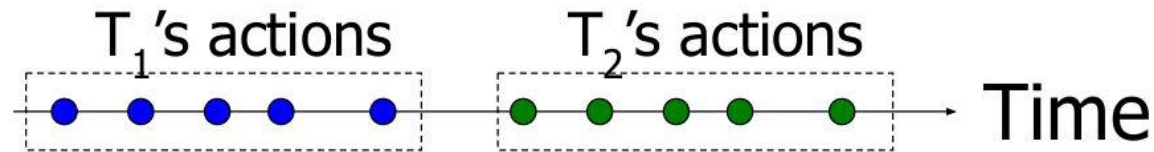
Schedule

- The exact sequence of (relevant) actions of one or more transactions.
- The database engine scheduler task is to produce interleaving of transactions steps that result in a consistent database state.

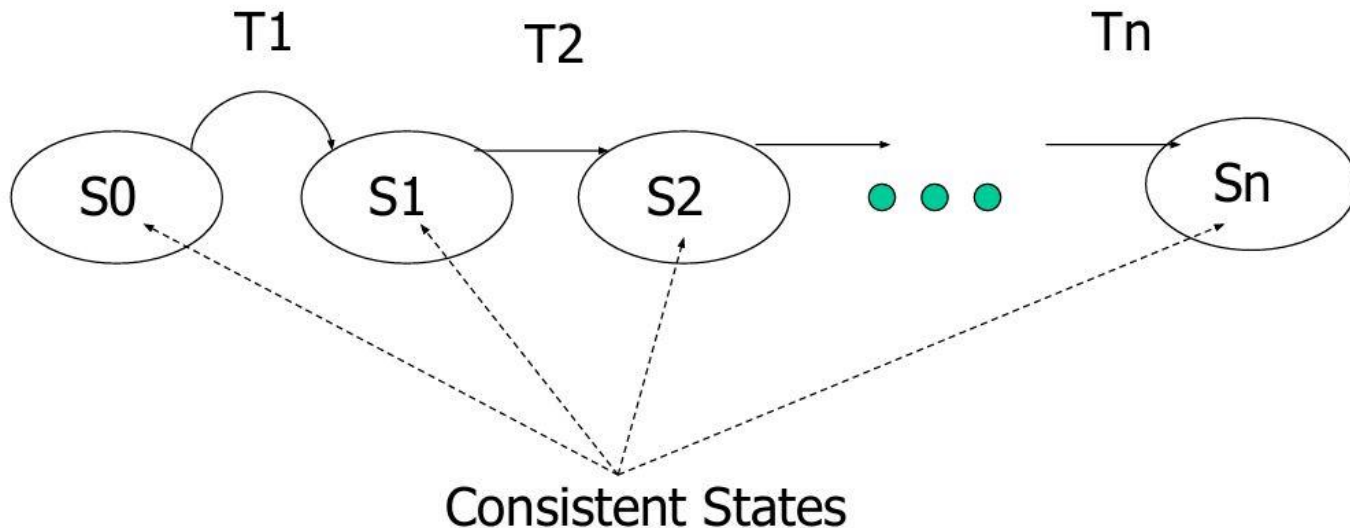


Serial Schedule

- If any action of transaction T_1 precedes any action of T_2 , then all actions of T_1 precede all actions of T_2 .
- No transaction starts unless the one before it ends.



- A serial schedule is the gold standard as it guarantees that the database is in a consistent state. However, it is too slow to use since it means running one transaction at a time, and thus no support for concurrency.



Serial Schedule 1:

Transaction T1	Transaction T2	A	B	C
Read (A, t) $t = t - 100$ Write (A, t) Read (B, t) $t = t + 100$ Write (B, t)		500	500	500
	Read (A, s) $s = s - 100$ Write (A, s) Read (C, s) $s = s + 100$ Write (C, s)	400	600	500
		300	600	600

$$300 + 600 + 600 = 1500$$

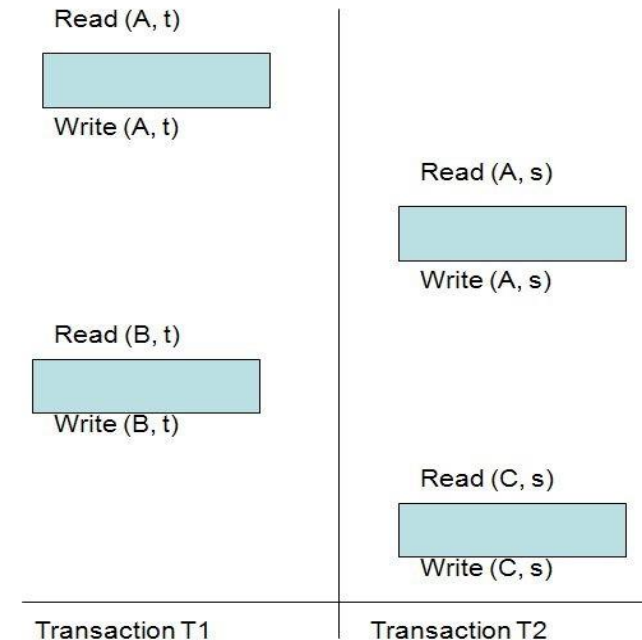
Serial Schedule :

Transaction T1	Transaction T2	A	B	C
		500	500	500
	Read (A, s) s = s -100 Write (A, s) Read (C, s) s = s + 100 Write (C, s)			
		400	600	500
Read (A, t) t = t - 100 Write (A, t) Read (B, t) t = t + 100 Write (B, t)				
		300	600	600

$$300 + 600 + 600 = 1500$$

Serializability

- A schedule is called **serializable** if its final effect is the same as that of a serial schedule.
- A **Serializable schedule** is fine and does not result in an inconsistent database, since serial schedules are fine.
- Non-serializable schedules are unlikely to result in consistent databases.
- Not possible to look at all $n!$ serial schedules to check if the effect is the same. Instead, we ensure serializability by allowing or not allowing certain schedules.
- We check **Conflict-Serializable** schedule produces the same result as one of the serial schedules. It is constructed by considering non-conflicting steps.
- Conflict Serializable schedules have a weaker notion of serializability, depends only on reads and writes, decide which steps can be interleaved and which cannot.



Conflict Serializable Schedule :

T1			T2			T1			T2		
read(A)						read(A)					
A = A - 50						A = A - 50					
write(A)						write(A)					
			read(A)								
			tmp = A*0.1			read(B)					
			A = A - tmp			B=B+50					
			write(A)			write(B)					
read(B)									read(A)		
B=B+50									tmp = A*0.1		
write(B)									A = A - tmp		
									write(A)		
			read(B)						read(B)		
			B = B+ tmp						B = B+ tmp		
			write(B)						write(B)		
Effect:	<u>Before</u>	<u>After</u>				Effect:	<u>Before</u>	<u>After</u>			
A	100	45	==			A	100	45			
B	50	105				B	50	105			

Conflict Serializable Schedule :

T1			T2		
read(A)					
A = A - 50					
write(A)					
			read(A)		
			tmp = A * 0.1		
			A = A - tmp		
			write(A)		
read(B)					
B = B + 50					
write(B)					
			read(B)		
			B = B + tmp		
			write(B)		
Effect:			Effect:		
	<u>Before</u>	<u>After</u>		<u>Before</u>	<u>After</u>
A	100	45		A	100
B	50	105	==	B	50
					105

Non-Conflict Serializable Schedule :

T1			T2			
read(A)						
A = A - 50						
write(A)						
			read(A)			
			tmp = A*0.1			
			A = A – tmp			
			write(A)			
read(B)						
B=B+50						
write(B)						
			read(B)			
			B = B+ tmp			
			write(B)			
Effect:	<u>Before</u>	<u>After</u>	! ==	Effect:	<u>Before</u>	<u>After</u>
A	100	45		A	100	45
B	50	105		B	50	55

Simple Notations

- $r_T(x) \rightarrow$ Transaction T reads X.
- $w_T(x) \rightarrow$ Transaction T writes X.
- Where X can be any element of the Database (Relation, Attribute, Tuple, Block,...)
- **Non-Conflicting Pairs:**
 - Two Reads: $r_i(X), r_i(Y)$.
 - Read and write of different database element: $r_i(X), w_j(Y)$.
 - Two writes of different database elements: $w_i(X), w_j(Y)$.
- **Conflicting Pairs:**
 - Two actions of the same transaction: $r_i(X), w_i(X)$.
 - Two writes of the same database element: $w_i(X), w_j(X)$.
 - A read and a write of the same database element: $r_i(X), w_j(X)$.

Conflict-Equivalent Schedule

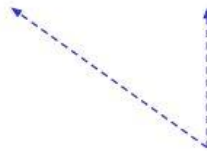
- If S can be transformed into S' through a series of swaps, S and S' are called **conflict-equivalent**.
- Conflict-equivalent guarantees the same final effect on the database.
- A schedule S is conflict-serializable if it is **conflict-equivalent** to a serial schedule.
- Turn a given schedule to a serial one by make as many nonconflicting swaps as we wish.

Precedence Graph

- Constructing a **precedence graph** over the transactions determines if it is conflict-serializable.
- A directed edge from T_1 and T_2 , if they have conflicting instructions, and T_1 's conflicting instruction comes first.
- If there is a cycle in the graph, then it is **not conflict-serializable**. (Can be checked in at most $O(n + e)$ time, where n is the number of vertices, and e is the number of edges).
- If there is none, then it is **conflict-serializable**.

Precedence graph for schedule S:

- Nodes: Transactions in S
- Edges: $T_i \rightarrow T_j$ whenever
 - S: ... $r_i(X)$... $w_i(X)$...
 - S: ... $w_i(X)$... $w_j(X)$...
 - S: ... $r_i(X)$... $w_j(X)$...



Note: not necessarily consecutive

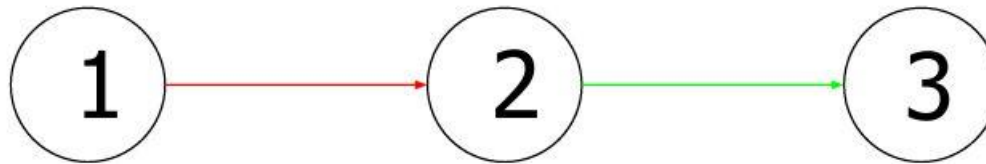
Precedence Graph

Precedence graph for schedule S

- Nodes: Transactions in S
- Edges: $T_i \rightarrow T_j$ whenever
 - S: ... $r_i(X)$... $w_i(X)$...
 - S: ... $w_i(X)$... $w_j(X)$...
 - S: ... $r_i(X)$... $w_j(X)$...

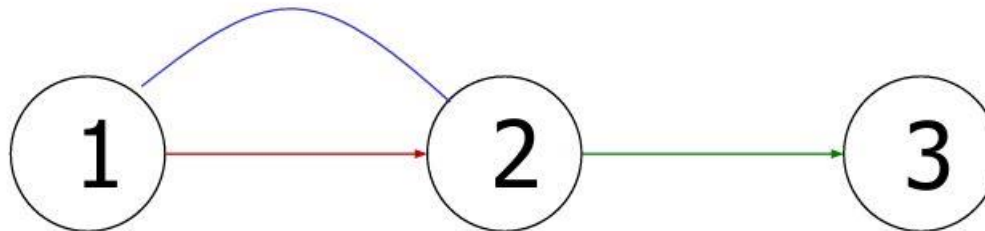
- Conflict Serializable:

$S_1: r_2(A); r_1(B); w_2(A); r_3(A); w_1(B); w_3(A); r_2(B); w_2(B);$



- Non-Conflict Serializable:

$S_2: r_2(A); r_1(B); w_2(A); r_2(B); r_3(A); w_1(B); w_3(A); w_2(B);$



Schedule Transformation

$r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B); r_2(B); w_2(B)$

- To transfer the schedule to a **Conflict-Equivalent** schedule, We swap all the **Non-Conflicting Steps**.

$r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B); r_2(B); w_2(B)$

$r_1(A); w_1(A); r_2(A); r_1(B); w_2(A); w_1(B); r_2(B); w_2(B)$

$r_1(A); w_1(A); r_1(B); r_2(A); w_2(A); w_1(B); r_2(B); w_2(B)$

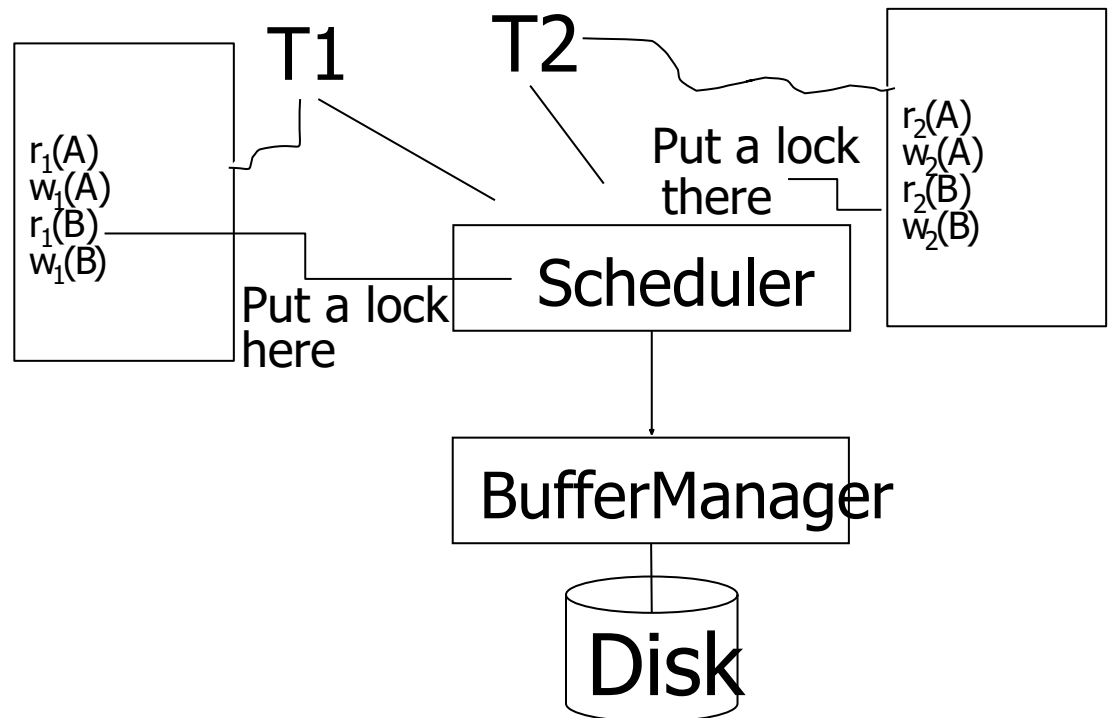
$r_1(A); w_1(A); r_1(B); r_2(A); w_1(B); w_2(A); r_2(B); w_2(B)$

$r_1(A); w_1(A); r_1(B); w_1(B); r_2(A); w_2(A); r_2(B); w_2(B)$

How to enforce serializable schedule

- **Option 1:** Run system any way, recording P(S); at end, check for P(S) cycles and declare if execution was good! (This solution is Unrealistic!).
- **Option 2:** Prevent P(S) cycles from occurring.
 - 2.1: Buffer transactions during n seconds, stop DBMS, make the schedule, execute the schedule, repeat. (Which is also unrealistic!).

2.2: Use Locks.



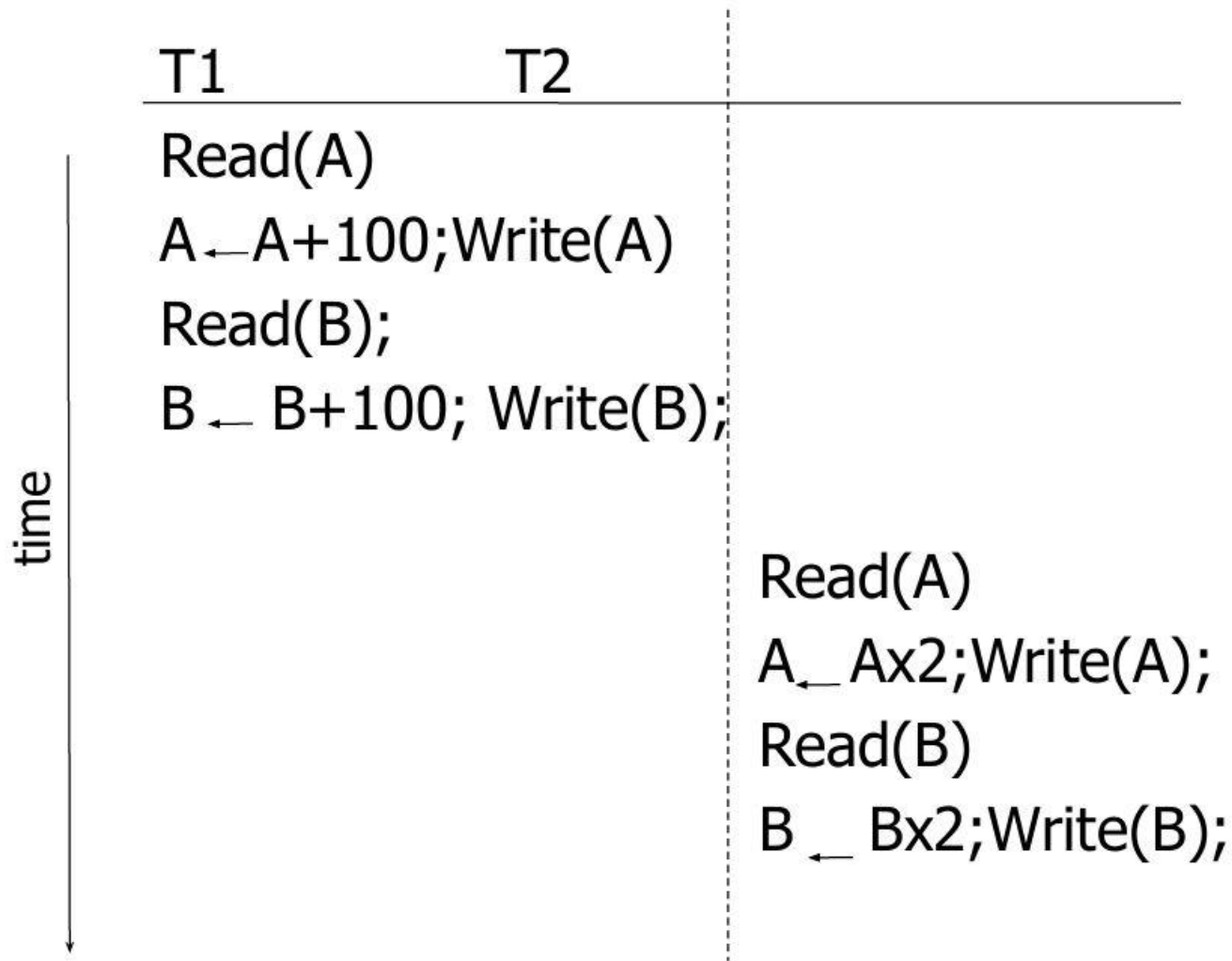
Locks

- Two Actions:
 - Lock: $l_i(A)$
 - Unlock: $u_i(A)$
- No transaction is allowed to lock any element that is locked by another transaction, before unlocking the element first.
- **Example:**
Insert locks for the following transactions:

T1: Read(A); A =A+100; Write(A);
 Read(B); B=B+100; Write(B);

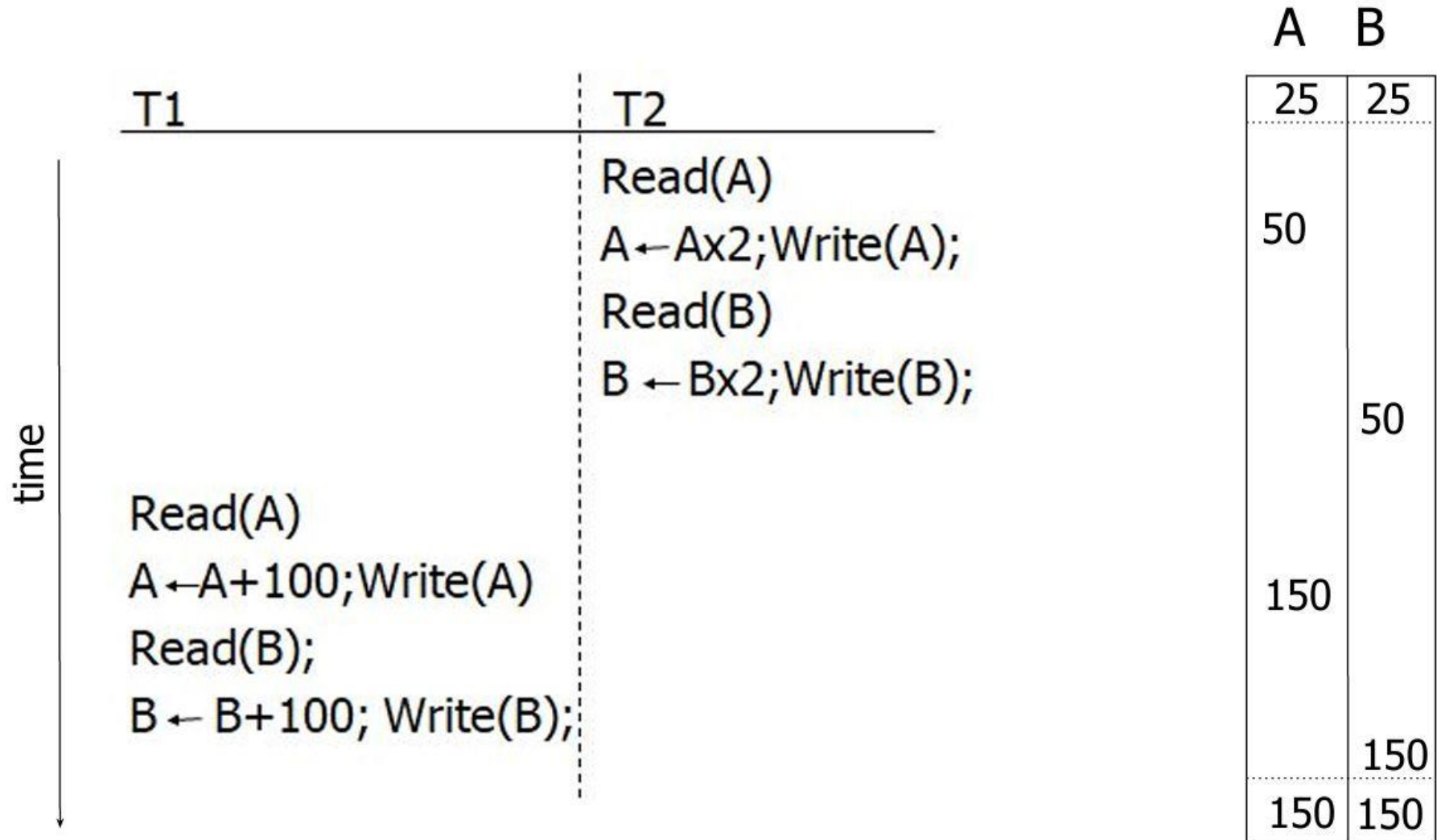
T2: Read(A); A =A*2; Write(A);
 Read(B); B=B*2; Write(B);

Serial Schedule of example



A	B
25	25
125	
	125
250	
	250
250	250

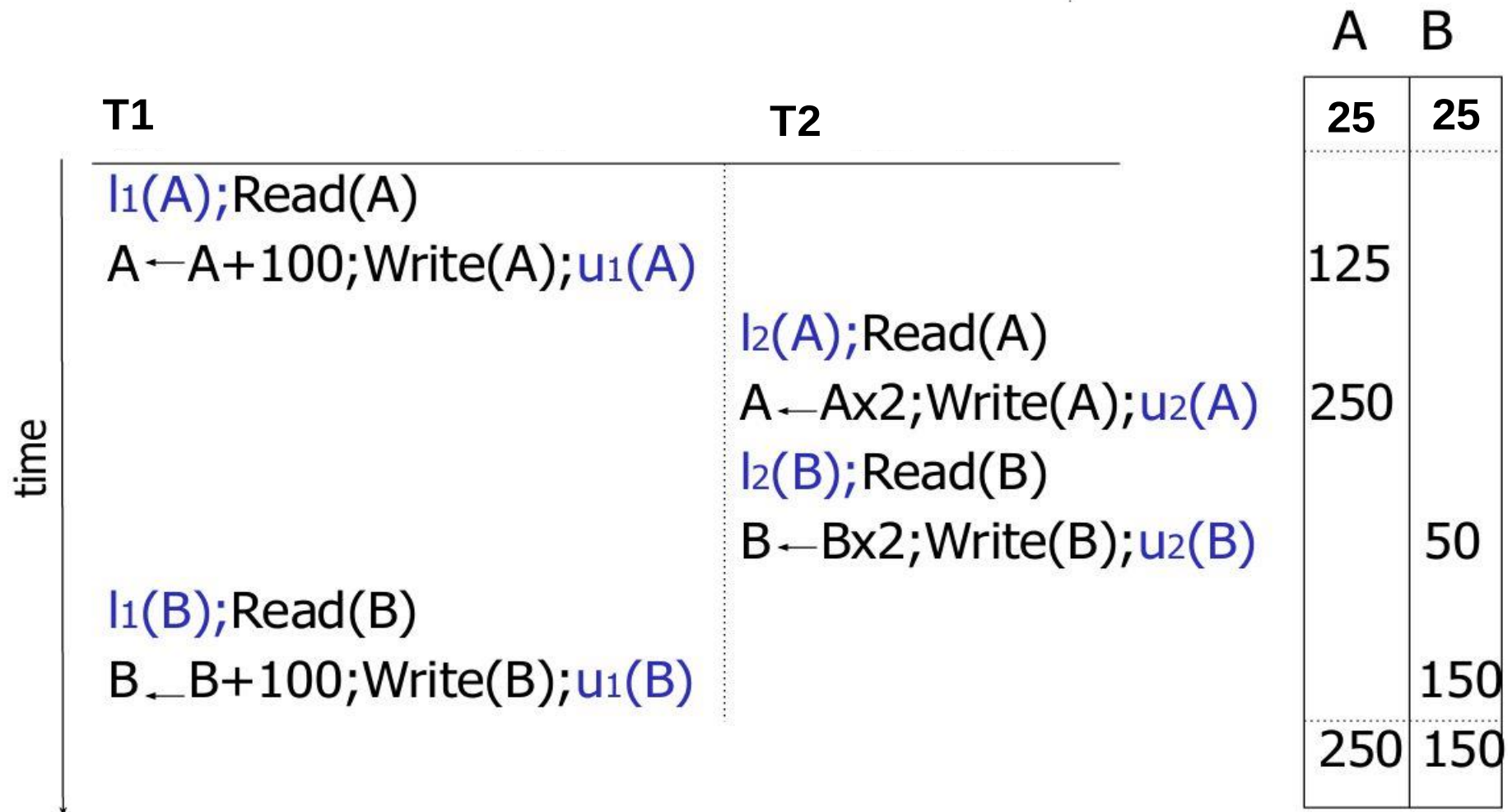
Serial Schedule of example



Adding Locks

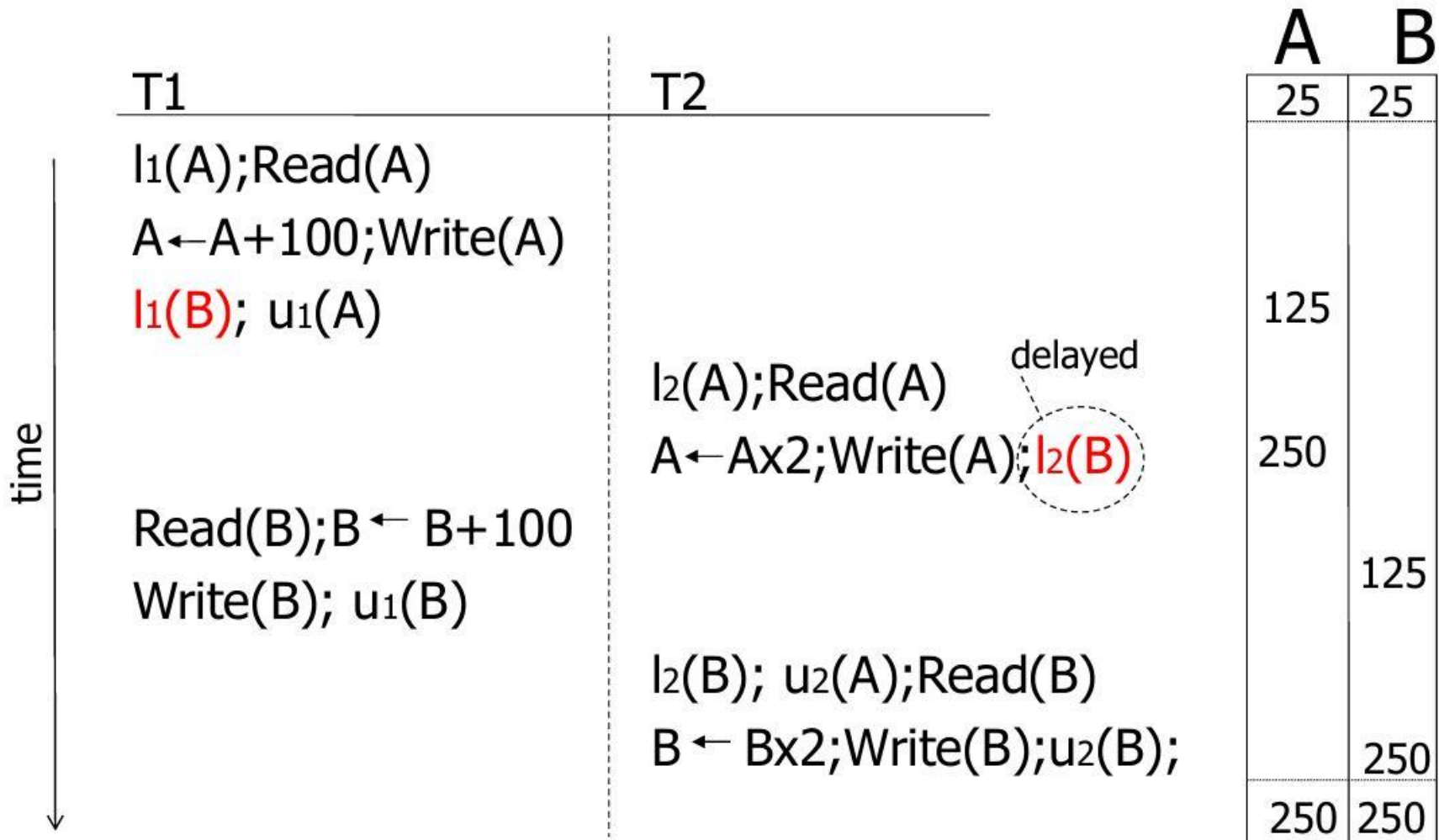
- A problem occurred !!

T1	T2
$l_1(A); \text{Read}(A)$	$l_2(A); \text{Read}(A)$
$A \leftarrow A + 100; \text{Write}(A); u_1(A)$	$A \leftarrow A \times 2; \text{Write}(A); u_2(A)$
$l_1(B); \text{Read}(B)$	$l_2(B); \text{Read}(B)$
$B \leftarrow B + 100; \text{Write}(B); u_1(B)$	$B \leftarrow B \times 2; \text{Write}(B); u_2(B)$



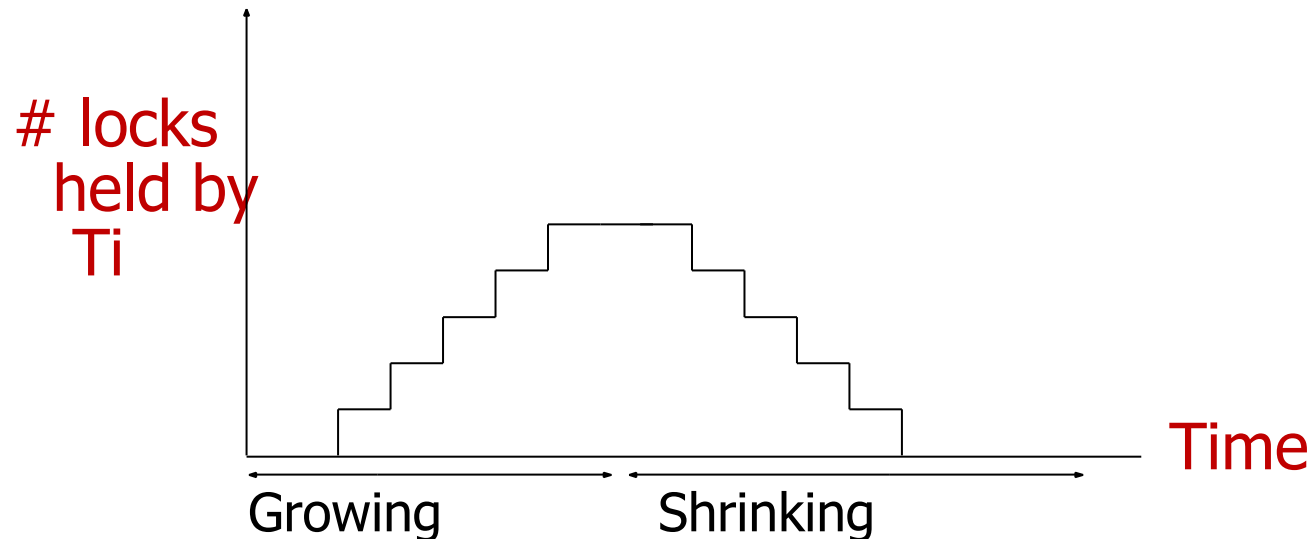
Two Phase Locking

- In every transaction, all lock actions precede all unlock actions:
 - First-phase locks are obtained.
 - Second-phase locks are released.



Two Phase Locking

- Serializability is enforced in database engines using two-phase locking.
- In two-phase locking, a transaction requests locks for all resources it needs access to before it releases any of those locks.
- Assume deadlocked transactions are rolled back:
 - They have no effect.
 - They do not appear in the schedule.
- Each transaction exhibit the below behavior.
- A surge in transactions count affects performance.
- It works but loads the OS and database engine.
- Must be used wisely.



Locking systems

		New request	
		S	X
Lock already held in	S	true	false
	X	false	false

- **Shared Lock:** When all transactions only **read** the data elements.

$$S = \dots l-S_i(A) \dots u_i(A) \dots$$

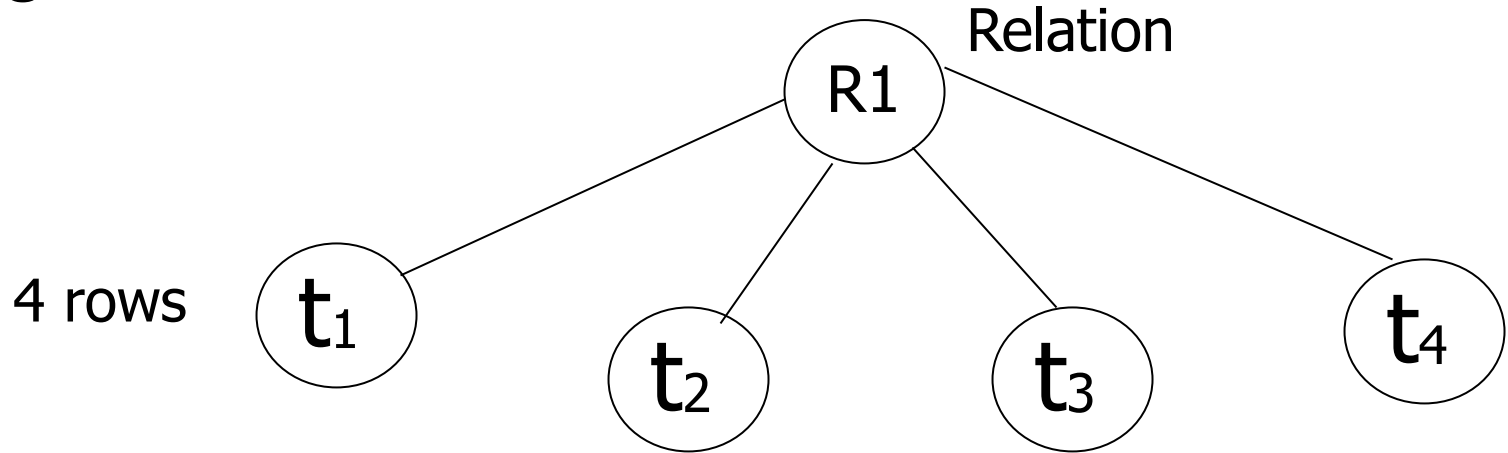
- **Exclusive Lock:** When a transaction writes to a data element.

$$T_i = \dots l-X_1(A) \dots r_1(A) \dots w_1(A) \dots u(A) \dots$$

- **Upgrading Locks:** first make the lock “shared”, till the transaction writes, then the lock is upgraded to “exclusive”

$$T_i = \dots l-S_1(A) \dots r_1(A) \dots l-X_1(A) \dots w_1(A) \dots u(A) \dots$$

Locking Tree



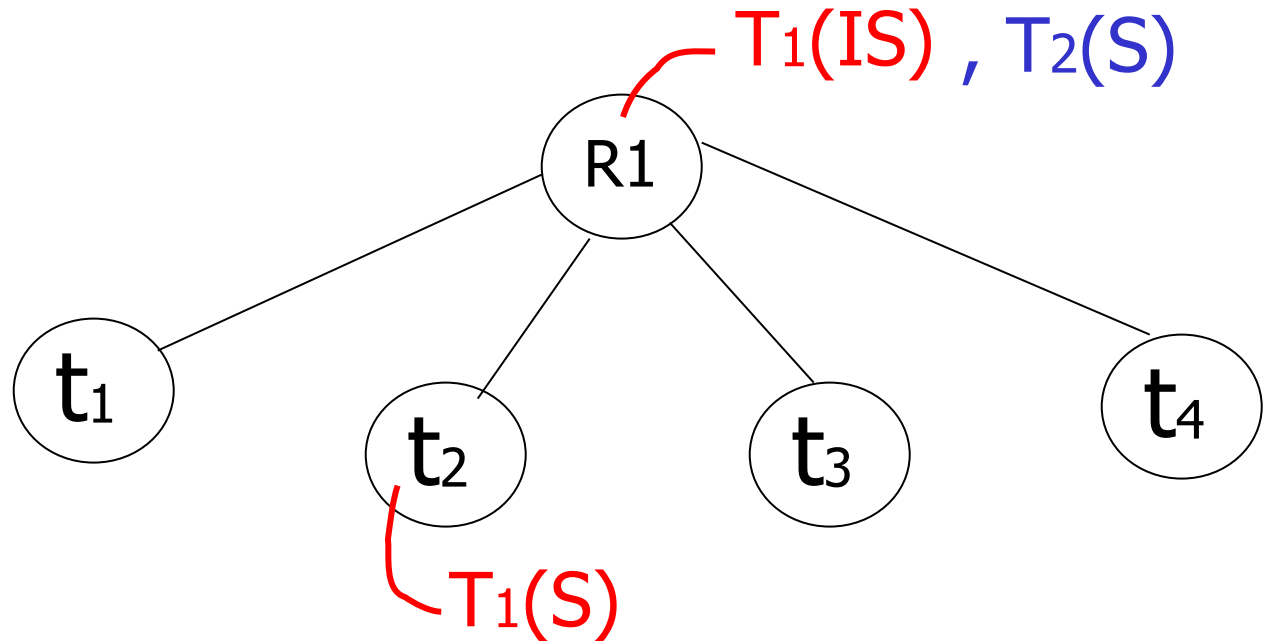
- A tree to maintain track of where locks are allocated.
- Have granular control of what is being locked.
- Contains the warning locks, denoted by the an 'I' (for intentional).

- **Algorithm:**

- Always begin at the root of the tree.
- If we are at the right element, request S or X lock.
- Else, request a warning lock IS or IX.
- If granted, move down the tree; else wait.

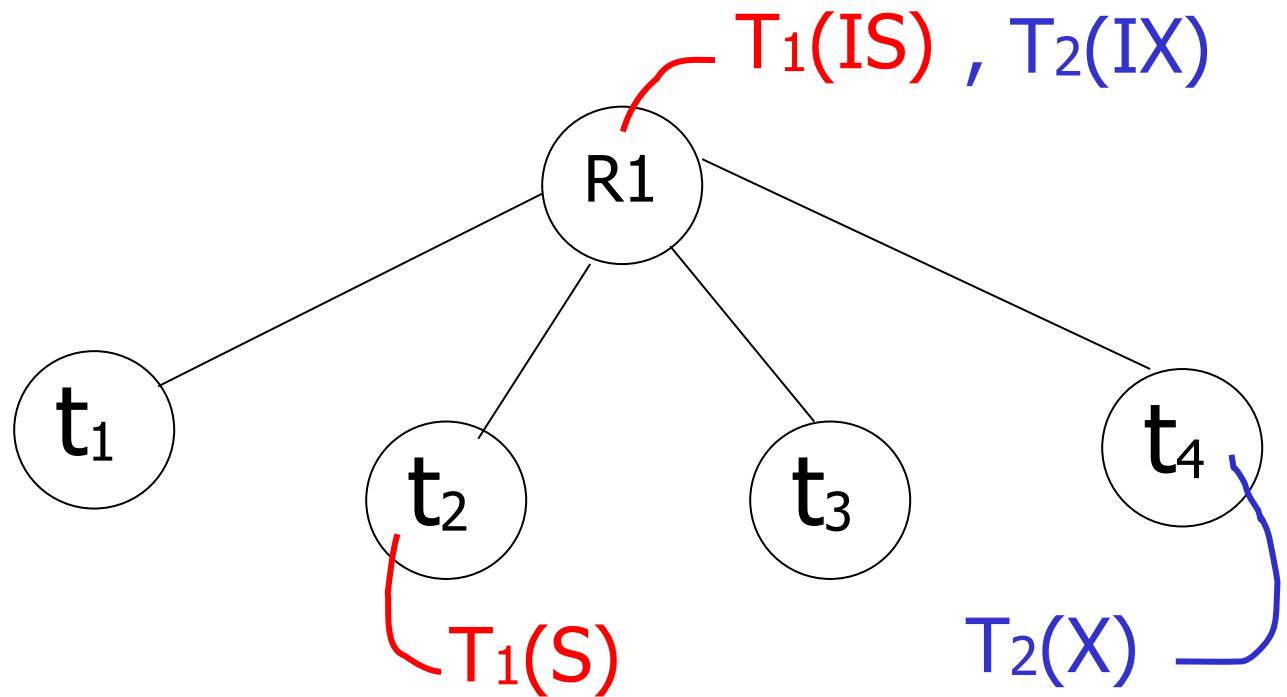
		Requestor			
		IS	IX	S	X
Holder	IS	T	T	T	F
	IX	T	T	F	F
	S	T	F	T	F
	X	F	F	F	F

Example 1



- T1 would like to read the row t2 (could be tuple).
- Starts from the root.
- Request intention to share (going to request later one a shared lock).
- If doesn't get it, will wait.
- If it will get it, will go down till t2.
- Another transaction T2 wants to write to a specific row t4.
- Gets a lock on t4 when reaches there.
- At the root request an intention exclusive.
- Move till t4 and gets exclusive on it.

Example 2



- T1 would like to read the row t2 (could be tuple).
- Starts from the root.
- Request intention to share (going to request later one a shared lock).
- If it doesn't get it, will wait.
- If it will get it, will go down till t2.
- Another transaction T2 wants to write to a specific row t4.
- Gets a lock on t4 when it reaches there.
- At the root, request an intention exclusive.
- Move till t4 and get exclusive on it.